# The AI Automation Blueprint for Solopreneurs

PRINTMAXX | printmaxxer.gumroad.com

# The AI Automation Blueprint: How to Run a Business While You Sleep

80+ automations I built for my solopreneur operation. Real scripts. Real results. Zero employees.

By PRINTMAXX

## What This Is

I run 80+ automated scripts that handle research, content, lead generation, outreach, analytics, and product distribution. Zero employees. Zero VAs. Just me, Python, and a handful of AI tools.

This document is the exact blueprint. Not theory. Not "you could do this." I DID this. Every automation described here exists and runs daily.

By the end of this guide, you will have a complete automation stack you can deploy in a weekend.

## Part 1: The Automation Mindset

### The Question That Changes Everything

Before building any automation, ask: "Will I do this more than 3 times?"

If yes, automate it. If no, do it manually and move on.

Most solopreneurs waste time automating things they'll do once. The real payoff comes from automating the things you do every single day.

### The 3 Layers of Automation

Layer 1: Cron Scripts (No AI Needed)
Simple Python scripts that run on a schedule. Scrape data, update spreadsheets, send alerts. These run 24/7 whether you're awake or not. Zero AI cost.

Layer 2: AI-Assisted Workflows
Scripts that use AI APIs (Claude, GPT, Gemini) for judgment calls. Content generation, lead scoring, alpha screening. These cost money per API call but save hours of human time.

Layer 3: Autonomous Agents
AI agents that can make decisions, take actions, and iterate without human input. These run overnight and produce deliverables by morning.

This guide covers all three layers. Start with Layer 1. It's free and handles 70% of what you need.

---

# Part 2: The Research Automation Stack

## The Problem

You need to monitor 200+ sources daily for opportunities. Twitter accounts, Reddit threads, competitor pricing, product trends, platform changes. Doing this manually takes 4-6 hours per day.

## The Solution: Automated Scraping Pipeline

Script 1: Reddit Scraper
Scrapes 40+ subreddits using Reddit's public JSON API. No login needed. No API key needed. Free.

How it works:

1. Reads a list of subreddits from a CSV file
2. Hits reddit.com/r/{subreddit}/hot.json for each one
3. Extracts posts with high engagement (>50 upvotes)
4. Filters for business-relevant keywords
5. Saves to a structured CSV with title, URL, score, comment count
6. Runs at 6:15 AM daily via cron

What you need:

- Python 3.x
- requests library (pip install requests)
- A CSV with subreddit names
- One cron entry: 15 6 * * python3 reddit_scraper.py

Subreddits worth monitoring for solopreneurs:
r/SideProject, r/EntrepreneurRideAlong, r/juststart, r/SaaS, r/Affiliatemarketing, r/Microconf, r/webdev, r/indiehackers, r/startups, r/smallbusiness

Script 2: Twitter Signal Scraper
Monitors 80+ high-signal Twitter accounts for new tactics, tools, and opportunities.

How it works:

1. Reads a list of Twitter handles from HIGH_SIGNAL_SOURCES.csv
2. Uses browser cookies from an already-logged-in session
3. Extracts tweets with engagement above threshold
4. Filters for business keywords vs noise
5. Saves to a staging CSV for review
6. Runs at 6:00 AM daily

High-signal accounts to monitor:

@levelsio (indie hacking, real revenue numbers), @tdinh_me (technical solopreneur), @dannypostmaa (honest failures), @marc_louvion (structured how-tos), @paborMbp (aggressive tactics), @cloakzy (content ops)

Script 3: Competitor Price Monitor
Monitors competitor pricing pages and alerts you when anything changes. Free alternative to Visualping ($5-50/month).

How it works:

1. Stores a snapshot of each monitored URL
2. Fetches the page daily
3. Computes a diff between old and new version
4. If change detected: saves the diff and sends an alert
5. Runs every 6 hours

Use cases:

- Monitor competitor SaaS pricing pages
- Track job postings (new hires = they have budget)
- Watch product pages for feature changes
- Monitor government contract portals

Script 4: Platform Algorithm Detection
Monitors TikTok, Instagram, and X/Twitter for algorithm changes that affect content reach.

How it works:

1. Checks platform developer blogs and status pages
2. Monitors creator community discussions about reach changes
3. Tracks hashtag performance over time
4. Alerts when engagement patterns shift significantly
5. Runs at 7:00 AM daily

## The Alpha Screening System

Raw scraper output is useless without scoring. The alpha screening system scores every finding on a 0-100 scale:

Scoring Breakdown (100 points total):

- Evidence Quality (30 pts): Hard evidence > text claims > no evidence
- Replicability (20 pts): Can you actually do this with your current tools?
- Time Decay (20 pts): How old is this alpha? Platform arbitrage decays 50%/month. App tactics decay 10%/month.
- Historical Performance (15 pts): Did similar tactics work before?
- ROI Potential (15 pts): HIGHEST vs HIGH vs MEDIUM vs LOW

Decision Thresholds:

- Score >= 70: SCALE (deploy with confidence)
- Score 50-69: PAPER_TRADE (test with minimal capital first)

- Score < 50: KILL (do not pursue)

Category-Specific Decay Rates:

- Platform arbitrage: 50% per month (windows close fast)
- Cold outbound: 20% per month (ESP rules tighten gradually)
- App factory: 10% per month (app tactics are stable)
- Growth hacks: 30% per month (platforms patch exploits)
- Algorithm trading: 40% per month (market edges decay fast)

This scoring system prevents you from chasing stale alpha. A tactic that scored 90 two months ago might score 45 today due to time decay.

---

## Part 3: The Content Automation Stack

### The Problem

You need to post across 6+ platforms daily. Writing unique content for each platform takes 3-4 hours.

### The Solution: Content Multiplication

Script 5: Content Multiplier (1-to-20)
Takes one piece of content and generates 20+ platform-specific variants.

Input: One blog post, thread, or research finding.

Output:

- 5 Twitter/X posts (hook + value, under 280 chars)
- 3 LinkedIn posts (professional angle, 1,300 char max)
- 1 Thread (5-7 tweets)
- 1 Newsletter intro paragraph
- 3 Instagram captions
- 2 Reddit posts (GEO-optimized for specific subreddits)
- 1 Medium article intro
- 1 Substack note
- 3 TikTok/Reels script hooks (15-30 seconds)

How to build this:

1. Define templates for each platform (character limits, tone, CTA style)
2. Extract key insights from source content
3. Reformat each insight into platform-native format
4. Add platform-specific CTAs
5. Output as separate files or a single CSV for bulk upload

Script 6: Buffer CSV Generator

Takes your content calendar and generates platform-ready CSVs for bulk upload to Buffer, Publer, or any scheduling tool.

How it works:

1. Reads a 30-day content calendar (date, time, platform, post text)
2. Splits into per-platform CSVs (Twitter, Instagram, LinkedIn, etc.)
3. Formats timestamps in each platform's required format
4. Adds hashtags from a hashtag library
5. Outputs upload-ready files

Script 7: Engagement Bait Converter
Takes research findings tagged as "good for engagement but not actionable strategy" and converts them into platform-native posts.

Example: A research finding says "indie hacking opportunities with AI are insane right now." That's engagement bait, not strategy. But it makes a great engagement post for your audience.

The converter:

1. Reads all ENGAGEMENT_BAIT tagged findings
2. Adapts each one for your niche and voice
3. Adds hooks, questions, or controversial takes
4. Outputs as ready-to-post content

### The 30-Day Content Calendar System

Build a 30-day content calendar once per month. Then the automation handles daily posting.

Structure per day:

- 8:00 AM: Value post (specific tip or number)
- 12:00 PM: Engagement post (question or hot take)
- 5:00 PM: Story post (what happened today, failure, lesson)

That's 90 posts per month per niche. With the content multiplier, one research session generates a full month of content.

---

## Part 4: The Lead Generation Automation Stack

### The Problem

Finding potential clients manually is slow. Researching each business, finding their email, assessing their website quality, writing personalized outreach -- it takes 30-60 minutes per lead.

### The Solution: Automated Lead Scoring Pipeline

Script 8: Lead Scraper with Scoring

Finds local businesses via DuckDuckGo search, visits their websites, and scores them 0-100 on website quality.

LOW scores = HOT leads. A business with a bad website is the perfect prospect for a website redesign service.

How it works:

1. Searches DuckDuckGo for "[industry] in [city]"
2. Extracts business names, addresses, phone numbers, websites
3. Visits each website and checks: mobile responsiveness, SSL, load speed, SEO tags, social links, contact forms, technology stack age
4. Scores each site 0-100 (lower = worse = hotter lead)
5. Outputs CSV: business_name, phone, website, score, signals, email_if_found

What you need:

- Python 3.x
- requests + beautifulsoup4 (pip install requests beautifulsoup4)
- No API keys, no browser automation, no Selenium

Script 9: Nationwide Scraper
Wraps the lead scraper to iterate across 200+ US cities automatically.

Input: Industry list + city list
Output: Master CSV with all leads from all cities, scored and ranked

Run this overnight. Wake up to 1,000+ scored leads across 10 cities.

Script 10: Mass Outreach Generator
Takes scored leads and generates personalized cold emails using industry-specific templates.

How it works:

1. Reads lead CSV (business name, score, signals detected)
2. Picks the right industry template (dental, legal, real estate, etc.)
3. Personalizes each email: business name, city, specific issue found
4. Generates 3-email sequence per lead (Day 0, Day 3, Day 7)
5. Outputs Instantly-compatible CSV for bulk sending

10 Industry Templates Built In:
Dental, Legal, Real Estate, SaaS, Agency, Coach, Ecommerce, Restaurant, Fitness, General

Each template has:

- Specific industry pain points
- ROI math relevant to their business size
- Breakup email pattern (respectful exit after 3 touches)
- Compliance language (CAN-SPAM, unsubscribe)

---

## Part 5: The Product Distribution Stack

### The Problem

You have digital products (ebooks, templates, courses) but listing them on 10+ marketplaces manually takes hours per product.

### The Solution: Multi-Platform Distributor

Script 11: Ecom Distributor
Takes one product spec and generates listings for 13 platforms.

Supported platforms:
Gumroad, Etsy, Redbubble, Creative Market, eBay, Amazon KDP, Lemon Squeezy, Whop, Payhip, Ko-fi, Sellfy, Notion Marketplace, AppSumo

How it works:

1. Reads product spec (title, description, price, tags, category)
2. Reformats for each platform's requirements
3. Generates platform-specific titles, descriptions, tags
4. Outputs upload-ready data (CSV or JSON per platform)
5. Optionally auto-lists via browser automation

Script 12: Auto-Lister
Uses Playwright (headless browser) to automatically list products on marketplaces where you're logged in.

Currently supports: Gumroad, Etsy, Redbubble, Fiverr

How it works:

1. Reads the listing data from the distributor
2. Opens the marketplace in a headless browser
3. Fills in the listing form
4. Uploads cover images
5. Submits the listing
6. Logs success/failure

When to use browser automation vs manual:

- First listing on a platform: manual (set up account, verify payment)
- Subsequent listings: automated (once logged in, bot handles the rest)

---

## Part 6: The Analytics and Tracking Stack

### The Problem

You have revenue from 10+ sources, expenses for tools and ads, and no way to see the full picture without opening 10 dashboards.

### The Solution: Unified Tracking

Script 13: Revenue Intake CLI

Log revenue from any source with one command:

```
python3 revenue_intake.py log --method "gumroad" --amount 47 --source "cold email subject lines"
```

Appends to a master CSV. Shows daily/weekly/monthly summaries. ASCII chart in terminal.

Script 14: Method Performance Analyzer

Analyzes all active revenue methods weekly. Identifies what's working, what's not, what to double down on.

Reads from: revenue tracker, expense tracker, time tracking, lead pipeline

Outputs: ranked methods by revenue/hour, recommendations, kill list

Script 15: Paper Trading System

Test new business methods with statistical rigor before deploying real capital.

How it works:

1. Define the method (cold email, app launch, content campaign)
2. Set a test budget ($50-100) and timeline (14 days)
3. Log every result: revenue, hours spent, leads generated
4. System calculates revenue/hour, confidence interval, Sharpe ratio
5. Decision: SCALE (>=70 score), ITERATE (50-69), or KILL (<50)

Minimum 10 data points before any SCALE/KILL decision. No gut feelings. No vibes. Data.

Script 16: Daily TODO Generator

Scans all systems at 8:30 AM and generates a prioritized daily task list:

1. What came in overnight (new scraper data, new leads)
2. What needs review (pending alpha, pending content)
3. What's ready to ship (products to list, content to post)
4. What's blocked (accounts to create, tools to set up)

Wake up. Read the TODO. Execute. No decision fatigue.

---

## Part 7: The Orchestration Layer (Cron)

All of the above runs automatically using cron (built into every Mac and Linux system).

### The Daily Schedule

```
2:00 AM  - Master overnight runner (all scrapers + analyzers)
6:00 AM  - Twitter signal scraper
6:15 AM  - Reddit subreddit scraper
7:00 AM  - Platform algorithm detection
7:15 AM  - Hashtag and audio tracking
```

```
8:00 AM  - Zero-cost opportunity scanner
8:30 AM  - Daily TODO generator
Every 6h - Alpha screening on new entries
Every 6h - Government contract monitor
6:00 PM  - Cross-system integration runner
9:00 PM  - Method performance analysis
```

### The Weekly Schedule (Monday)

```
3:00 AM  - Platform RPM tracking
3:30 AM  - Creator program monitoring
4:00 AM  - ASO keyword research
4:30 AM  - Government tenders refresh
5:00 AM  - Lead generation (10 cities x 5 industries)
5:15 AM  - Trending products scan
```

### How to Set Up Cron

On Mac/Linux, open terminal:

```
crontab -e
```

Add entries in this format:

```
MINUTE HOUR * * * cd /your/project && python3 script.py >> logs/script.log 2>&1
```

Example: Run reddit scraper at 6:15 AM daily:

```
15 6 * * * cd /home/user/project && python3 reddit_scraper.py >> logs/reddit.log 2>&1
```

That's it. The script now runs every day at 6:15 AM whether you're awake or not.

---

## Part 8: The 3-Layer Architecture

### Layer 1: Always Running (No AI)

These scripts use only Python standard library + requests. Zero AI cost. Zero API keys. Run 24/7.

| Script | What It Does | Frequency |
|---|---|---|
| Reddit scraper | Monitors 40+ subreddits | Daily |
| Competitor monitor | Tracks pricing changes | Every 6h |
| Lead scraper | Finds and scores businesses | Weekly |
| Revenue tracker | Logs and analyzes income | On demand |
| TODO generator | Prioritizes daily tasks | Daily |
| Performance analyzer | Ranks methods by ROI | Weekly |

### Layer 2: AI-Assisted (API Cost)

These use Claude/GPT APIs for judgment. ~$5-30/month in API costs depending on volume.

| Script | What It Does | API Cost |
|---|---|---|
| Alpha screening | Scores research findings | ~$0.10/batch |
| Content multiplier | 1 piece to 20 variants | ~$0.50/batch |
| Email personalizer | Customizes outreach | ~$0.02/email |

### Layer 3: Autonomous Agents (Claude Max)

These are AI agents that run for extended periods, making decisions and producing deliverables.

| Agent | What It Does | Duration |
|---|---|---|
| Research agent | Scans sources, extracts alpha | 30-60 min |
| Content agent | Generates week of content | 20-30 min |
| Audit agent | Reviews all systems for gaps | 15-20 min |

---

## Part 9: Build Your Stack This Weekend

### Day 1 (Saturday): Foundation

Morning (2 hours):

1. Create project folder structure
2. Set up Python virtual environment
3. Install dependencies: pip install requests beautifulsoup4
4. Create your subreddit list CSV (start with 10 subreddits)
5. Create your high-signal Twitter accounts CSV (start with 20 accounts)

Afternoon (3 hours):

6. Build and test reddit scraper (run against 3 subreddits)
7. Build and test lead scraper (run against 1 city, 1 industry)
8. Build and test competitor price monitor (add 5 competitor URLs)
9. Set up log directory

Evening (1 hour):

10. Install cron entries for all 3 scripts
11. Verify first overnight run
12. Create revenue tracking CSV with headers

### Day 2 (Sunday): Content + Outreach

Morning (2 hours):

1. Build content multiplier (start with Twitter format only)
2. Create 30-day content calendar template

3. Generate first week of content from one research finding

Afternoon (3 hours):

4. Build mass outreach generator (start with 1 industry template)
5. Run lead scraper on 5 cities
6. Generate outreach emails for top-scored leads
7. Set up Instantly.ai or similar for bulk sending

Evening (1 hour):

8. Build daily TODO generator
9. Run full system test
10. Review tomorrow's auto-generated TODO

### Monday Morning

Wake up. Check the TODO. Your scrapers ran overnight. You have new research data. Your content calendar has posts queued. Your lead pipeline has scored prospects.

Now you just execute.

---

## Part 10: Common Pitfalls

1. Don't automate what you haven't done manually first. If you've never sent a cold email, don't build an automation system. Send 50 emails manually. Learn what works. THEN automate.

2. Don't over-engineer. Your first version should be ugly. 50 lines of Python that works beats 500 lines that's "clean." Ship, then refine.

3. Don't automate sensitive actions. Payments, account creation, publishing to social media with your real accounts: do these manually until you trust the system.

4. Respect rate limits. DuckDuckGo: 1 request per 2 seconds. Reddit JSON: 1 request per 2 seconds. If you hammer APIs, you get blocked.

5. Log everything. Every script writes to a log file. When something breaks at 3 AM, the log tells you what happened.

6. Start with cron, not Airflow. You don't need a job orchestration framework. You need crontab -e. Simple works.

7. Don't buy tools for problems you can solve with 20 lines of Python. Visualping is $5-50/month. A competitor monitor script is 80 lines of Python and $0/month.

8. Test on small data first. Run the lead scraper on 1 city before running it on 200. Run the content multiplier on 1 piece before running it on 100.

---

# Appendix A: Python Cheat Sheet for Non-Programmers

If you've never written Python, here's enough to get started.

Install Python:
Mac: brew install python3 or download from python.org
Windows: Download from python.org, check "Add to PATH"

Install a library:

```
pip3 install requests beautifulsoup4
```

Run a script:

```
python3 my_script.py
```

Read a CSV:

```
import csv
with open("data.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["column_name"])
```

Make a web request:

```
import requests
response = requests.get("https://example.com")
print(response.text)
```

Save to a file:

```
with open("output.txt", "w") as f:
    f.write("Hello world")
```

Schedule with cron (Mac/Linux):

```
crontab -e
# Add this line to run script.py every day at 6 AM:
0 6 * * * python3 /full/path/to/script.py >> /full/path/to/log.txt 2>&1
```

That's it. You now know enough Python to build automations.

---

# Appendix B: Tool Recommendations

| Category | Free Option | Paid Option | When to Upgrade |
|---|---|---|---|
| Scraping | Python requests + bs4 | ScrapingBee, Browserless | When you hit rate limits |
| Email sending | Gmail (50/day limit) | Instantly.ai ($37/mo) | When you have 100+ leads |
| Content scheduling | Manual posting | Buffer ($6/mo) | When posting 3+ times/day |
| Revenue tracking | CSV files | Stripe dashboard | When revenue > $1K/mo |
| Analytics | Google Analytics (free) | Plausible ($9/mo) | When privacy matters |

| | | | |
|---|---|---|---|
| AI APIs | Claude free tier | Claude Pro / API | When you need volume |
| Browser automation | Playwright (free) | Browserbase | When you need cloud execution |

---

Built by PRINTMAXX. This is the exact system I run. Not theory. Not "you could." I did.

Get the full Solopreneur Ops System (cron templates, all script templates, CSV schemas, deployment guide): printmaxxer.gumroad.com

---

Disclaimer: Results not typical. Individual results vary based on effort, market conditions, and other factors.